

Capitolo 32 — PROT-003 — Direct Before Delegate

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-32
Data master	2026-08-30
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/32_prot_003_direct_before_delegate.md
Source SHA	3837b4e
Release	2026-08-31T04:04:44.305Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-08-30 **Scope:** WCM generale, domain-agnostic

32.0 Prima di delegare, chiedersi se serve davvero

Delegare può sembrare automaticamente una buona idea. Se esiste un altro agente, un servizio o uno strumento specializzato, perché non affidargli il lavoro?

Nel WCM la risposta è: perché ogni passaggio aggiuntivo ha un costo organizzativo. Aumenta il numero di transizioni, può introdurre attese, può perdere contesto e può trasformare una persona in un semplice intermediario tra sistemi che avrebbero potuto comunicare direttamente.

PROT-003 — Direct Before Delegate nasce per evitare questo spreco.

La sua regola fondamentale è:

| Usare la capacità minima necessaria e più diretta disponibile.

In parole semplici: **se l'azione può essere eseguita direttamente, nel runtime corrente e nel rispetto della governance, non si introduce un intermediario senza una ragione reale.**

Questo non significa che delegare sia sbagliato. Significa che la delega deve risolvere un limite concreto, non diventare un riflesso automatico.

32.1 Il problema che PROT-003 risolve

Immaginiamo un esempio pedagogico e astratto.

Un sistema deve leggere un documento conservato in un archivio al quale ha già accesso. Potrebbe leggerlo direttamente. Invece chiede a un secondo sistema di aprirlo, poi quel secondo sistema invia il contenuto a una persona, e infine la persona lo riporta al primo sistema.

Il risultato finale può anche essere corretto. Ma per ottenere la stessa informazione sono stati aggiunti passaggi che non hanno fornito una capacità nuova.

Ogni passaggio superfluo può produrre:

- maggiore latenza;
- maggiore consumo di risorse;
- più occasioni di errore;
- perdita o deformazione del contesto;
- maggiore dipendenza dall'intervento umano;
- più difficoltà nel capire chi abbia realmente eseguito e verificato l'azione.

PROT-003 affronta precisamente questo problema: **prima di delegare, distingue ciò che può essere fatto direttamente da ciò che richiede davvero un'altra capacità.**

32.2 Delegare non è il comportamento predefinito

Nel WCM la delega non è un valore in sé.

È una conseguenza di un limite reale di:

- capacità tecnica;
- accesso;
- runtime;
- località dell'ambiente;
- governance.

Questa distinzione è importante perché un'organizzazione agentica può diventare inefficiente anche quando ogni singolo componente funziona correttamente. Basta che le attività vengano instradate attraverso troppi passaggi.

PROT-003 cerca quindi il percorso operativo più diretto compatibile con ciò che serve davvero.

La domanda iniziale è semplice:

| Posso eseguire questa azione direttamente adesso?

Se la risposta è sì, e nessuna regola di governance lo impedisce, l'azione resta diretta.

Se la risposta è no, il WCM deve capire **perché**.

32.3 Il trigger

Il protocollo si attiva quando un'azione sta per essere instradata verso un altro service, agente o ambiente operativo, oppure quando si sta per concludere che la capacità necessaria non esiste.

Il trigger non è quindi “esiste un service”.

Il trigger è:

C'È UN'AZIONE DA ESEGUIRE

↓

SI STA DECIDENDO CHI O COSA DEVE ESEGUIRLA

↓

PROT-003

Il protocollo interviene **prima** che la delega venga assunta come soluzione.

32.4 Gli input necessari

Per classificare correttamente l'azione servono pochi elementi, ma devono essere reali.

Occorre conoscere:

- quale azione concreta deve essere eseguita;
- quali capacità sono disponibili nel runtime corrente;
- quali accessi sono realmente disponibili;
- se una parte del lavoro richiede un ambiente locale o esterno;
- quali service autorizzati possono fornire una capacità mancante;
- quali vincoli di governance limitano l'esecuzione diretta o delegata.

Un punto è particolarmente importante: **la disponibilità di una capacità e l'autorizzazione a usarla non sono la stessa cosa.**

Un sistema può tecnicamente essere in grado di eseguire un'azione e, nello stesso tempo, non avere authority per farlo.

PROT-003 governa il routing operativo; non elimina i gate di governance.

32.5 Il prerequisito quando la risposta sembra essere “no”

Dire “non posso farlo direttamente” è una conclusione operativa importante, perché può provocare una delega, uno stop o la registrazione di un capability gap.

Per questo il WCM non permette che quel **NO** derivi soltanto dalla memoria o da una supposizione.

Quando la classificazione dipende dall'assenza di una capacità diretta, entra in gioco **PROT-011 – Capability Evidence Check Before Block.**

In linguaggio semplice:

prima di dire che una capacità non c'è, bisogna verificarlo nel runtime corrente quando esiste un meccanismo adatto per farlo.

Questo evita un errore molto comune: trasformare una vecchia informazione, una mancata visibilità iniziale o un problema temporaneo nella falsa convinzione che una capacità sia strutturalmente assente.

PROT-011 distingue infatti almeno quattro situazioni:

- capacità disponibile;

- capacità presente ma temporaneamente bloccata;
- capacità realmente assente anche dopo la verifica del fallback autorizzato;
- capacità non verificabile con evidenza sufficiente.

Questa verifica sostiene PROT-003 quando serve una risposta negativa. Non è una nuova authority e non autorizza automaticamente l'azione.

32.6 Le quattro classificazioni operative

Dopo aver chiarito l'azione e, quando necessario, verificato la capability, PROT-003 classifica il routing in quattro categorie.

32.6.1 DIRECT

DIRECT significa che la capacità necessaria è disponibile direttamente nel runtime corrente.

Per esempio, in un caso astratto, se il sistema può già leggere un documento remoto mediante uno strumento disponibile, non serve chiedere a un secondo attore di leggerlo e riportarne il contenuto.

L'azione corretta è:

| eseguire direttamente, salvo un vincolo di governance contrario.

DIRECT non significa “fare tutto da soli”. Significa soltanto che, per quella specifica azione, aggiungere un intermediario non produce capacità utile.

32.6.2 LOCAL_REQUIRED

LOCAL_REQUIRED indica che l'azione richiede qualcosa che esiste soltanto in uno specifico ambiente locale non accessibile direttamente dal runtime corrente.

Un esempio pedagogico: verificare lo stato reale di un file che esiste esclusivamente su una determinata macchina fisica.

Qui la delega ha una ragione concreta: l'altro ambiente possiede un accesso che il runtime corrente non possiede.

La parte locale viene quindi affidata al service appropriato e autorizzato.

32.6.3 SERVICE_REQUIRED

SERVICE_REQUIRED significa che l'azione non è eseguibile direttamente e richiede una capacità specialistica o operativa esterna disponibile attraverso un service autorizzato.

La regola resta quella della minima sufficienza: **si attiva il service minimo necessario a superare il limite reale.**

Non si trasferisce più lavoro del necessario soltanto perché il service è disponibile.

32.6.4 CAPABILITY_GAP

CAPABILITY_GAP è la situazione più forte: la capacità non è disponibile direttamente e non esiste un service validato e autorizzato che possa fornirla nel perimetro corrente.

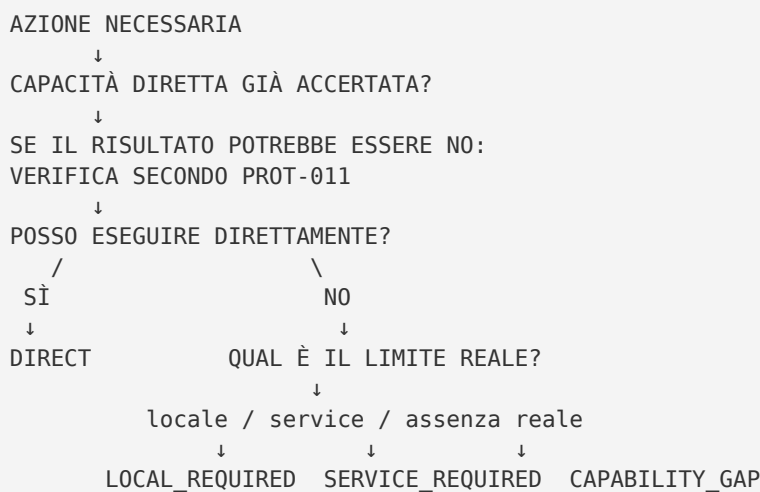
Questa conclusione non può essere inventata né dedotta automaticamente da un errore temporaneo.

Quando applicabile deve essere sostenuta dalla Capability Evidence Check di PROT-011.

Se la capacità non può essere verificata con sufficiente evidenza, l'incertezza resta tale: non viene trasformata artificialmente in un gap certo.

32.7 Il flusso completo

Il protocollo può essere letto come una sequenza di decisioni.



La sequenza non cerca di massimizzare il numero di componenti coinvolti.

Cerca di ottenere il risultato con **il percorso più diretto che conservi capacità, authority e verificabilità necessarie**.

32.8 Il gate prima della delega

Prima di delegare o dichiarare un capability gap, il protocollo richiede di poter rispondere a quattro domande.

1. **Posso farlo direttamente adesso?**
2. **Se la risposta è no, come è stata verificata quando PROT-011 è applicabile?**
3. **Qual è il limite concreto: assenza, blocco temporaneo o necessità locale?**
4. **Qual è il service minimo che supera quel limite?**

Queste domande costituiscono il cuore decisionale del protocollo.

Se alla prima domanda la risposta è sì, l'azione resta diretta salvo governance contraria.

Se la risposta è no, il routing deve riflettere la natura reale del limite, non una scorciatoia organizzativa.

32.9 Un task può essere diviso

Una delle conseguenze più importanti di PROT-003 è che un task non deve essere necessariamente classificato tutto nello stesso modo.

Immaginiamo un'attività composta da due parti:

- leggere e analizzare dati disponibili direttamente;
- verificare un elemento conservato soltanto in un ambiente locale.

Sarebbe possibile delegare l'intera attività al componente locale. Ma così si delegherebbe anche ciò che non richiede quella capacità.

PROT-003 preferisce la scomposizione:

```
TASK
├─ parte eseguibile direttamente → DIRECT
└─ parte che richiede ambiente locale → LOCAL_REQUIRED
```

In questo modo il service esterno viene usato per ciò che aggiunge realmente, mentre il resto del lavoro conserva continuità e contesto nel percorso diretto.

L'esempio è pedagogico: non introduce una nuova regola, ma rende visibile la regola di escalation già presente nel protocollo canonico.

32.10 Un blocco temporaneo non è una capacità assente

Supponiamo che uno strumento necessario esista, ma in quel momento non sia utilizzabile per un problema di autenticazione, un limite temporaneo o un errore tecnico.

Sarebbe scorretto concludere immediatamente:

| *“La capacità non esiste.”*

Il problema riguarda l'utilizzabilità contingente, non necessariamente l'esistenza strutturale della capacità.

PROT-011 formalizza questa distinzione e PROT-003 la eredita nel routing.

Per il lettore non tecnico, l'analogia è semplice: una porta chiusa perché la serratura è momentaneamente guasta non dimostra che la stanza non esista.

Questa distinzione protegge il WCM da false stop condition e deleghe costruite su diagnosi sbagliate.

32.11 Gli output del protocollo

PROT-003 non produce necessariamente un documento. Produce prima di tutto **una classificazione operativa del routing**.

L'output può essere:

- **DIRECT;**
- **LOCAL_REQUIRED;**

- `SERVICE_REQUIRED`;
- `CAPABILITY_GAP`.

Quando la conclusione dipende da una capacità negativa, l'evidenza raccolta secondo PROT-011 rende ricostruibile perché quella classificazione sia stata scelta.

Il valore dell'output non sta quindi solo nell'etichetta. Sta nel fatto che l'organizzazione può capire **perché l'azione è rimasta diretta, perché è stata delegata oppure perché non è disponibile una capacità sufficiente.**

32.12 I failure mode

Il protocollo esiste perché alcune deviazioni sono particolarmente dannose.

Delegare ciò che è già DIRECT

È il caso classico: il sistema possiede già accesso e capacità, ma introduce comunque un intermediario.

Il risultato può essere corretto, ma il percorso è inutilmente più lungo e fragile.

Assumere che una capability manchi senza verificarla

Una capacità non visibile immediatamente o non disponibile in una run precedente viene trattata come assente oggi.

Questo può generare deleghe o stop falsi.

Confondere blocco temporaneo e capability gap

Un problema di autenticazione, permesso, rate limit, outage o errore tecnico viene interpretato come assenza strutturale della capacità.

Delegare l'intero task quando soltanto una parte richiede un service

In questo caso il limite reale di una singola componente viene esteso artificialmente a tutto il lavoro.

Confondere capability con authority

Il sistema scopre di poter tecnicamente eseguire un'azione e conclude, erroneamente, di essere anche autorizzato a farlo.

PROT-003 non conferisce questa authority.

32.13 Perché ridurre gli intermediari migliora la continuità

Ogni passaggio tra componenti può richiedere una traduzione del contesto: cosa bisogna fare, cosa è già noto, quali limiti esistono, quale risultato deve tornare indietro.

Quando un passaggio non aggiunge una capacità reale, questa traduzione diventa puro overhead organizzativo.

Ridurre le deleghe inutili aiuta quindi a conservare:

- continuità del contesto;
- tracciabilità dell'esecuzione;
- chiarezza delle responsabilità;
- minore necessità di intervento umano come ponte tra sistemi.

Il protocollo collega questa idea al Human Intervention Ratio: l'obiettivo operativo è massimizzare risultato e continuità per ogni intervento umano realmente necessario.

Non significa eliminare l'umano. Significa evitare di usarlo come “bus” quando i componenti possono operare direttamente o attraverso un service appropriato.

32.14 Relazioni con gli altri elementi WCM

PROT-003 lavora insieme ad altri elementi del WCM senza sostituirli.

La relazione più diretta è con **PROT-011 – Capability Evidence Check Before Block**.

```
PROT-011
verifica se la capacità è realmente disponibile
      ↓
PROT-003
sceglie il routing operativo appropriato
```

PROT-011 fornisce il prerequisito epistemico quando il routing dipende da una risposta negativa. PROT-003 usa quella evidenza per distinguere direct, necessità locale, necessità di service e gap reale.

PROT-003 resta inoltre subordinato alla governance applicabile. La disponibilità tecnica non sostituisce authority, gate, scope o contratti operativi.

Infine, la classificazione prodotta può influenzare il modo in cui un processo o un workflow prosegue, ma il protocollo non ridefinisce da solo il lifecycle del lavoro.

32.15 Maturity e limiti

Il protocollo canonico PROT-003 è dichiarato **VALIDATED**.

Questa qualifica appartiene alla baseline corrente del protocollo e riflette l'evidenza disponibile nel WCM. Non deve essere letta come dimostrazione universale che ogni organizzazione, runtime o configurazione produca automaticamente gli stessi benefici.

Il protocollo ha inoltre limiti chiari.

Non stabilisce:

- che l'esecuzione diretta sia sempre preferibile indipendentemente dalla governance;
- che ogni delega sia inefficiente;

- che la presenza tecnica di uno strumento equivalga ad autorizzazione;
- che un errore temporaneo debba essere ignorato;
- che ogni task debba essere scomposto oltre ciò che è utile e proporzionato.

Il suo campo è più preciso: **impedire deleghe superflue e classificare il routing sulla base della capacità realmente disponibile e del limite realmente esistente.**

32.16 Source map

Il capitolo deriva dalla baseline canonica corrente e usa soltanto le fonti necessarie al suo perimetro:

- `WCM_AGENT_START.md` — bootstrap e principi operativi applicabili;
- `wcm/documentation/process-memory-book/BOOK_INDEX.md` — mapping editoriale CH32 → PROT-003;
- `wcm/process-book/protocols/PROT-003_DIRECT_BEFORE_DELEGATE.md` — fonte tecnica primaria;
- `wcm/process-book/protocols/PROT-011_CAPABILITY_EVIDENCE_CHECK_BEFORE_BLOCK.md` — fonte collegata necessaria per il prerequisite di evidenza.

`BOOK_STATUS.md` resta bookkeeping editoriale derivato e non viene usato come fonte tecnica del protocollo.

32.17 La regola da ricordare

Se tutto il capitolo dovesse essere ridotto a una sola idea, sarebbe questa:

Prima di delegare, verifica se la capacità necessaria è già disponibile direttamente; se non lo è, delega soltanto ciò che serve a superare il limite reale.

Il WCM non misura la qualità dell'orchestrazione dal numero di agenti coinvolti.

La misura dalla capacità di usare **il percorso più diretto, sufficiente, verificato e autorizzato** per arrivare al risultato.